49th IEEE Conference on Decision and Control
December 15-17, 2010
Hilton Atlanta Hotel, Atlanta, GA, USA

Strategy Optimization with Its Application to Dynamic Games

Daizhan Cheng, Yin Zhao, Yifen Mu

***Abstract*— A metric space structure on the set of finite memory strategy profiles is proposed. The geometric meaning of this metric about the network structure is revealed. Based on this metric, a numerical method called hill climbing method is proposed to find the local "best" strategy profile. Two applications of this technique are introduced. First, we consider a dynamic game. The local Nash/sub-Nash equilibriums are defined, and then the problem of finding Nash/sub-Nash solutions to finite games over the set of μ-memory strategies are investigated. Second, the optimization of mixed-value logical dynamic control systems is studied, where the the strategies with initial conditions are considered. Several examples are included to demonstrate the efficiency of the method.**

I. Introduction

Game theory has wide applications to economics, evolutionary biology, ecology, engineering, political and international relations, military conflicts, etc. Finding equilibriums is the fundamental task for solving the game-based problems. We refer to [11] for basic concepts and notations.

Let G be a game with finite players and each player has finite possible actions. Such a game is called a finite game. Now if G is repeated infinite times, we call it the infinitely repeated (or dynamic) game of G and denote it by G_∞. This paper considers such dynamic games. Moreover, the feasible strategies of the game are assumed to be depending on finite histories. Then the strategies become a mixed-value logical dynamic system.

Boolean network is the simplest logical dynamic system. It was proposed by Kauffman in 1969 [14] to describe the genomic regularity network. Since then, it has attracted a great attention from the biologists, physicians, and system scientists. The structure and properties of Boolean networks and their applications to systems biology have been studied in many literatures, e.g., [1], [2], [10], [12], [13], [20], [21]. [15] and [16] are two nice introduction books.

Recently, using the semi-tensor product of matrices and the matrix expression of logic [3], a new technique for analyzing and synthesizing the logical dynamic (control) systems has been developed [4], [5], [6], [7]. This method has also been used to get Nash/sub-Nash solution for dynamic games [18], [9].

A major difficulty in searching Nash/sub-Nash solution is the computation complexity. In this paper, instead of searching global Nash/sub-Nash equilibriums we look for local ones. The basic idea in this paper is the following: We first assume the strategies are initial-value-independent and give a metric space structure to the set of strategy profiles. Then we show that the topology induced by such a metric is closely related to the network structure. Then we develop a numerical method, similar to hill climbing method in the optimization of multi-variable functions, to find local Nash/sub-Nash solutions. Then we consider the optimization of mixed-value logical dynamic systems. Particularly, it can be considered as a particular player in a game to find his best strategy. An algorithm is developed for searching optimal strategy, which is initial-value-depending.

This work was supported partly by SIC07010201 and NNSF 60674022, 60736022 of China.

Daizhan Cheng, Yin Zhao and Yifen Mu are with Lab. of Systems and Control, AMSS, Chinese Academy of Sciences, Beijing 100190, P.R.China. E-mail: dcheng@iss.ac.cn, zhaoyin@amss.ac.cn, mu@amss.ac.cn

II. Preliminaries

To give a precise description for the games and strategies concerned in the paper, we first introduce some notations:

- $\mathcal{D}_k := \{1, 2, \cdots, k\}$.
- δ_k^i: the i-th column of the identity matrix I_k.
- $\Delta_k := \{\delta_k^i | i = 1, \cdots, k\}$.
- A matrix $L \in M_{n\times r}$ is called a logical matrix if the columns of L, denoted by $\mathrm{Col}(L)$, have the form of δ_n^k. That is, $\mathrm{Col}(L) \subset \Delta_n$.
  Denote by $\mathcal{L}_{n\times r}$ the set of $n \times r$ logical matrices.
- If $L \in \mathcal{L}_{n\times r}$, by definition it can be expressed as $L = [\delta_n^{i_1}, \delta_n^{i_2}, \cdots, \delta_n^{i_r}]$. For the sake of compactness, it is briefly denoted as $L = \delta_n[i_1, i_2, \cdots, i_r]$. Its column set can also be denoted as $Col(L) = \delta_n\{i_1, i_2, \cdots, i_r\}$.
- A matrix $L \in M_{n\times r}$ is called a Boolean matrix if its entries are either 0 or 1. Denote by $\mathcal{B}_{n\times r}$ the set of $n \times r$ Boolean matrices.

The games considered in this paper are described as follows [9]

Definition 2.1: 1) A static finite game G consists of three ingredients:

(i) n players, named $A_1, \cdots, A_n$;
(ii) each player A_i has k_i possible actions, denoted by $x_i \in \mathcal{D}_{k_i}$, $i = 1, \cdots n$;
(iii) n payoff functions for n players respectively as

$$\begin{aligned} &c_j(x_1 = i_1, \cdots, x_n = i_n) \\ =&c^j_{i_1\ i_2\ \cdots\ i_n}, \quad j = 1, \cdots, n. \end{aligned} \tag{1}$$

2) A set of actions of every players, denoted by $s = (x_1, \cdots, x_n)$, is called a (pure) strategy profile of G, the set of strategy profiles is denoted by S.

In this paper we investigate the infinitely repeated game, G_∞, of a static game G.

Definition 2.2: Consider the infinitely repeated game G_∞ of G.

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on December 31,2025 at 08:10:04 UTC from IEEE Xplore. Restrictions app

1) A strategy profile is

$$s = (s_1, \cdots, s_n), \tag{2}$$

where s_j is a sequence of logical functions of time, called the strategy of player j, precisely,

$$s_j = \left\{ x_i(0), s_j^t \middle| \, i = 1, \cdots, n, t = 1, 2, \cdots \right\}$$

where s_j^t is a function of historical actions, precisely

$$x_j(t) = s_j^t(x_1(0), \cdots, x_n(0), \cdots, x_1(t-1), \cdots, x_n(t-1)).$$

Denote by S_∞ the set of strategy profiles.

2) The corresponding payoff functions for G_∞ are the averaged payoffs defined as [19], [18]

$$J_j(s) = \overline{\lim_{T\to\infty}} \frac{1}{T} \sum_{t=1}^{T} c_j(x_1(t), \cdots, x_n(t)). \tag{3}$$

For dynamic game G_∞, we are particularly interested in finite-memory strategies, which are defined as follows[9]:

Definition 2.3: Consider the dynamic game G_∞ of G. A μ-memory strategy is a strategy, where the action $x_j(t+1)$ depends on the past μ historical actions. Precisely, the strategy is generated by

$$x_j(t+1) = f_j(x_1(t), \cdots, x_n(t), \cdots, x_1(t-\mu+1), \cdots, x_n(t-\mu+1)), \tag{4}$$

with initial values

$$x_j(t) = x_j^t, \quad t \le \mu - 1, \ j = 1, \cdots, n. \tag{5}$$

Equivalently, we can also denote the set of initial values as

$$X^0 = \left\{ x_1^0, \cdots, x_n^0, \ \cdots, \ x_1^{\mu-1}, \cdots, x_n^{\mu-1} \right\}.$$

Remark 2.4:

1) Throughout this paper we consider only finite memory strategies. That is, $0 < \mu < \infty$. The set of μ-memory strategy profile is denoted by S_∞^μ.

2) Denote

$$k := \prod_{j=1}^{n} k_j.$$

It is easy to see that the number of total possible strategy profile for static game G is k.

3) In (4) f_j is a mapping $\underbrace{(\mathcal{D}_{k_1} \times \cdots \times \mathcal{D}_{k_n}) \times \cdots \times (\mathcal{D}_{k_1} \times \cdots \times \mathcal{D}_{k_n})}_{\mu} \to \mathcal{D}_{k_j}$, $j = 1, \cdots, n$.

4) Since the number of possible actions for G is finite, the actions under each μ-memory strategy profile will converge to either a fixed point or a cycle, which are called an attractor of the strategy profile dynamics. Precisely, denote

$$X(t) = (x_1(t), x_2(t), \cdots, x_n(t)),$$

a cycle is a loop

$$l := \{X(\alpha), X(\alpha+1), \cdots, X(\alpha+T) = X(\alpha)\}$$

with minimum period, the minimum period is called the length of the cycle, and length-1 cycle is also called fixed point.

5) Assume the attractor of a strategy profile s is

$$C := \{X(\alpha), X(\alpha+1), \cdots, X(\alpha+T) = X(\alpha)\}.$$

Then the payoff functions (3) becomes

$$J_j(s) = \frac{1}{T} \sum_{\beta=0}^{T-1} c_j(X(\alpha+\beta)).$$

Particularly, for any finite-memory strategy, the "$\overline{lim}$" in payoff functions (3) becomes "$\lim$".

A μ-memory strategy profile, concerned in this paper, is generated by the logical dynamical equation (4) with initial values (5). When $k_j = 2$, $j = 1, \cdots, n$, (4) becomes a Boolean network[14], [15], [16]. Recently, using semi-tensor product "$\ltimes$" of matrices, which extends the conventional matrix product of $A_{m\times n}$ and $B_{p\times q}$ from satisfying $n = p$ to any arbitrary n and p [3], certain basic structure properties of Boolean network have been revealed[7], [4]. When $k_j = \kappa > 2$, $\forall j$, we refer to [17] for the corresponding results. We also refer to [5], [6] for the control of Boolean networks. To use the matrix expression of logical equations, a logical variable must be expressed as a vector. Recall that we briefly denote the actions x_j by integers as $x_j \in \{1, 2, \cdots, k_j\}$. To use vector and matrix expressions for logical variables and/or mappings, we identify

$$i \sim \delta_{k_j}^i, \quad i = 1, \cdots, k_j; \ j = 1, \cdots, n.$$

Then we have

$$x_j \in \Delta_{k_j}, \quad j = 1, \cdots, n.$$

Set $x = \ltimes_{j=1}^n x_j$. Then

$$x \in \Delta_k.$$

The main results for unified $k_j = \kappa$, $\forall j$ can easily be extended to different k_j. Particularly, (4) can be expressed into its component-wise algebraic form as (refer to [9] for the details of the transformation)

$$x_j(t+1) = L_j \ltimes_{i=0}^{\mu-1} x(t-i), \quad j = 1, \cdots, n, \tag{6}$$

where

$$L_j \in \mathcal{L}_{k_j \times k^\mu}, \quad j = 1, \cdots, n.$$

Multiplying both sides of the equations in (6) and applying some properties of semi-tensor product to simplify it, we then can get the integrated algebraic form as[9]

$$x(t+1) = L^s \ltimes_{i=0}^{\mu-1} x(t-i), \quad j = 1, \cdots, n, \tag{7}$$

where

$$L^s \in \mathcal{L}_{k \times k^\mu}.$$

It has been proved that[9] (4), (6), and (7) are all equivalent, and it is easy to convert them from one form to another. We simply call L^s the structure matrix of the corresponding strategy profile s.

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on December 31,2025 at 08:10:04 UTC from IEEE Xplore. Restrictions app

III. Initial-Value-Independent Nash/Sub-Nash Equilibriums

From the discussion in Section 1 one sees that a μ-memory strategy profile, s, is uniquely determined by a logical matrix $L^s \in \mathcal{L}_{k\times k^\mu}$ and a set of initial values, X^0, which is described in (5). So we can simply denote $s=(L^s,X^0)$.

As long as L^s is determined, there are only finite number of attractions, say, $C_1,\cdots,C_\ell$. Starting from each set of initial values X^0, the trajectory will converge to a unique attractor. Then the set of sets of initial values is partitioned into J_α, $\alpha=1,\cdots,\ell$, such that $X^0\in J_\alpha$ implies the trajectory, starting from X^0, converges to C_α. Recall the argument in Remark 2.4, it is clear that if $X^0,\tilde{X}^0$ belong to the same J_α, then

$$J_j(L^s;X^0)=J_j(L^s;\tilde{X}^0),\quad j=1,\cdots,n.$$

Note that $L^s\in\mathcal{L}_{k\times k^\mu}$ is uniquely corresponding to $(L^s_1,L^s_2,\cdots,L^s_n)$, where $L^s_j\in\mathcal{L}_{k_j\times k^\mu}$ is the structure matrix of f_j, which is the strategy of player j.

In this section we are interested in the initial value independent strategies. We define the common Nash equilibrium.

Definition 3.1: A strategy profile

$$L^{s*}=(L^{s*}_1,L^{s*}_2,\cdots,L^{s*}_n)$$

is called a common Nash equilibrium, if it, combining with any set of initial values, is a Nash equilibrium. Precisely, $\forall j=1,\cdots,n$,

$$\begin{array}{l}J_j(L^{s*}_1,\cdots,L^{s*}_j,\cdots,L^{s*}_n;X^0)\geq\\ J_j(L^{s*}_1,\cdots,L^{s}_j,\cdots,L^{s*}_n;X^0)\\ \forall L^s_j\in\mathcal{L}_{k_j\times k^\mu},\forall X^0\in\prod_{i=1}^n\mathcal{D}^\mu_{k_i}.\end{array}\tag{8}$$

If the common Nash equilibrium does not exist, we may look for a sub-Nash solution. To make it precise, we give the following definition.

Definition 3.2: 1) Given a strategy profile

$$L^s=(L^s_1,L^s_2,\cdots,L^s_n).$$

Then we can find a non-negative real number $\epsilon^s\geq 0$, such that, $\forall j=1,\cdots,n$,

$$\begin{array}{l}J_j(L^s_1,\cdots,L^s_j,\cdots,L^s_n;X^0)+\epsilon^s\geq\\ J_j(L^s_1,\cdots,L^{s'}_j,\cdots,L^s_n;X^0),\\ \forall L^{s'}_j\in\mathcal{L}_{k_j\times k^\mu},\ \forall X^0\in\prod_{i=1}^n\mathcal{D}^\mu_{k_i}.\end{array}\tag{9}$$

The smallest $\epsilon^s\geq 0$, satisfying (9), is called a tolerance of s.

2) L^s is called a sub-Nash equilibrium if

$$\epsilon^s\leq\epsilon^{s'},\quad\forall s'\in S.\tag{10}$$

Hereafter till Section 5 we consider only the common Nash/sub-Nash equilibrium. Then we have the set of μ-memory strategies the same as the corresponding set of logical functions. Precisely,

$$S^\mu_\infty=\mathcal{L}_{k\times k^\mu},\quad\mu=1,2,\cdots.\tag{11}$$

Only Section 6 considers the initial-depending strategies.

IV. Metric Space Structure of Strategy Profile

To search for local optimal strategies, we need to establish a metric space structure for strategic space. The vector distance has been defined over Boolean vectors [22] and generalized to Boolean matrices [8]. To introduce it, we give the following definition.

Definition 4.1: Let $A=(a_{ij}),B=(b_{ij})\in\mathcal{B}_{m\times n}$. Then

1) For an anary logical operator $\sigma:\mathcal{B}_{m\times n}\to\mathcal{B}_{m\times n}$, defined as $\sigma A:=(\sigma a_{ij})$.
2) For a binary logical operator $\sigma:\mathcal{B}_{m\times n}\times\mathcal{B}_{m\times n}\to\mathcal{B}_{m\times n}$, defined as $A\sigma B:=(a_{ij}\sigma b_{ij})$.

The vector distance between two Boolean matrices is defined as follows.

Definition 4.2: Let $A=(a_{ij}),\ B=(b_{ij})\in\mathcal{B}_{m\times n}$. Then the vector distance between A and B, denoted by $\delta(A,B)$, is defined as

$$\delta(A,B)=A\bar{\vee}B.\tag{12}$$

The vector distance has some basic properties, which follow from the definition immediately.

Proposition 4.3: Let $A=(a_{ij}),\ B=(b_{ij}),\ C=(c_{ij})\in\mathcal{B}_{m\times n}$, $E\in\mathcal{B}_{p\times m}$, and $F\in\mathcal{B}_{n\times q}$. Then

1) δ formally satisfies the basic properties of a distance.
2)

$$\delta(EAF,EBF)\leq E\delta(A,B)F.\tag{13}$$

When $E\in\mathcal{L}_{p\times m}$ and $F\in\mathcal{L}_{n\times q}$, the inequality (13) becomes equality.

Note that as a convention, $A\geq B$ ($A>B$) means $a_{i,j}\geq b_{i,j}$ ($a_{i,j}>b_{i,j}$) $\forall i,j$.

If $A=(a_{ij})\in\mathcal{B}_{m\times n}$, we denote

$$\|A\|=\sum_{i=1}^m\sum_{j=1}^n a_{i,j}.\tag{14}$$

Next, we define a distance for two Boolean matrices of the same dimension.

Definition 4.4: Let $A=(a_{ij}),B=(b_{ij})\in\mathcal{B}_{m\times n}$. Then the distance between A and B, denoted by $d(A,B)$, is defined as

$$d(A,B):=\frac{1}{2}\|\delta(A,B)\|.\tag{15}$$

The following result is an immediate consequence of the definition.

Theorem 4.5: $(\mathcal{B}_{m\times n},d)$ is a metric space. That is,

(i)

$$d(A,B)=0\Leftrightarrow A=B,\quad\forall A,B\in\mathcal{B}_{m\times n};\tag{16}$$

(ii)

$$d(A,B)=d(B,A),\quad\forall A,B\in\mathcal{B}_{m\times n};\tag{17}$$

(iii)

$$d(A,C)\leq d(A,B)+d(B,C),\quad\forall A,B,C\in\mathcal{B}_{m\times n}.\tag{18}$$

Now we use this d to the set of μ-memory strategy profiles. It follows that $(\mathcal{L}_{k\times k^\mu},d)$ is a metric subspace of $(\mathcal{B}_{k\times k^\mu},d)$. We would like to explore what is the physical meaning of

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on December 31,2025 at 08:10:04 UTC from IEEE Xplore. Restrictions app

this d on the set of μ-memory strategy profiles. We denote by $\mathrm{Col}_i(A)$ the i-th column of A, and $|S|$ the cardinal number of set S (for a finite set, it is the number of the elements in S). Then the following proposition is an immediate consequence of the definition.

Proposition 4.6: Let $A, B \in \mathcal{L}_{k\times k^\mu}$. Then

$$|\{i\,|\mathrm{Col}_i(A) \neq \mathrm{Col}_i(B)\}| = d(A,B). \tag{19}$$

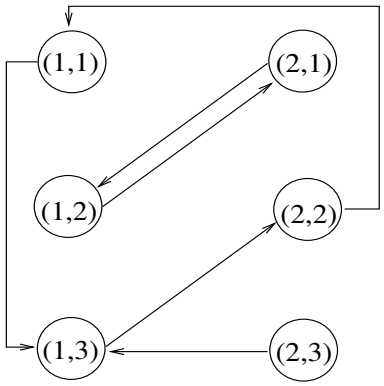
When $\mu = 1$, we have more clear description for the geometric meaning of this distance. Now assume $\mu = 1$ and L is given, the next action $x(t+1)$ can be determined from $x(t)$, then we can draw a strategy dynamic graph to depict L. We give an example for this.

Example 4.7: Assume a game consists of two players A and B. A has two actions and B has three actions, i.e., $x_1 \in \Delta_2$ and $x_2 \in \Delta_3$. Setting $x = x_1x_2$, we assume the strategy profile is

$$x(t+1) = Lx(t) = \delta_6[3,4,5,2,1,3]x_1(t)x_2(t). \tag{20}$$

Note that $\delta_6^1 \sim (1,1)$, $\delta_6^2 \sim (1,2)$, $\delta_6^3 \sim (1,3)$, $\delta_6^4 \sim (2,1)$, $\delta_6^5 \sim (2,2)$, $\delta_6^6 \sim (2,3)$, according to (20) it is easy to draw the strategy dynamic graph as in Fig. 1.

Fig. 1 Strategy Dynamic Graph



The following theorem shows the deep geometric meaning of the distance.

Theorem 4.8: Assume $\mu = 1$. Then the distance $d(L_1, L_2)$ between two strategy profiles is the number of different edges between the strategy dynamic graphs of L_1 and L_2.

Proof. In the strategy dynamic graph each node has exactly out-degree 1, that is, there is exactly one directed edge starting from each node. For a strategy profile L assume $\mathrm{Col}_i(L) = \delta_k^j$, then it means the strategy profile will lead $x(t) = \delta_k^i$ to $x(t+1) = \delta_k^j$. That is, $\mathrm{Col}_i(L)$ uniquely determines the edge starting from the node δ_k^i. The conclusion follows. □

As for $\mu > 1$ case, since $x(t+1)$ depends on μ historical strategy profiles, we define a path

$$x(t-\mu+1) \to x(t-\mu+2) \to \cdots \to x(t) \to x(t+1)$$

as a compounded edge. The strategy dynamic graph of a μ-memory strategy profile consists of all such compounded edges. Then the following corollary is clear.

Corollary 4.9: Assume $\mu > 1$. Then the distance $d(L_1, L_2)$ between two μ-memory strategy profiles is the number of different compounded edges between the strategy dynamic graphs of L_1 and L_2.

V. Local Nash/Sub-Nash Equilibriums

Consider the set of μ-memory strategy profiles, S_∞^μ. Let $L_0 \in \mathcal{L}_{k\times k^\mu}$. $\epsilon \in \mathbb{Z}_+$ be a positive integer. The ϵ-neighborhood of L_0 is defined as

$$B_\varepsilon(L_0) = \{\, L \in S^\mu |\, d(L, L_0) \leq \varepsilon \}. \tag{21}$$

Definition 5.1: A strategy profile L_0 is called a local Nash/sub-Nash equilibrium on B_ϵ, if it is a Nash/sub-Nash equilibrium with respect to its ϵ-neighborhood.

We explain the motivation for investigating local Nash/sub-Nash equilibriums. It was shown in [9] that the number of μ-member strategy profiles is

$$|\mathcal{L}_{k\times k^\mu}| = k^{(k^\mu)}. \tag{22}$$

It is clear that finding the Nash/sub-Nash equilibriums by exhaustive search is extremely difficult except some simplest cases. To find the Nash/sub-Nash solutions for a dynamic game, our idea is (1) Build a proper topological structure (metric) on the set of strategy profiles. This topology should be meaningful, which means the neighboring strategy profiles are essentially related. (2) Under this structure we can search a local Nash/sub-Nash solution by certain optimization process. In fact, (1) has been solved in previous section. As for (2) we may use hill climbing method. Then we propose the following algorithm. In the algorithm we consider a Nash equilibrium as a special sub-Nash equilibrium with tolerance $\epsilon = 0$. In the following the sub-Nash equilibrium is either common sub-Nash one or mean sub-Nash one.

Before stating the algorithm, we give some notations:

(i) $\mathcal{H}$: the set of historic strategy profiles, which have been used before. Denote by $\mathcal{H}^c = \mathcal{L}_{k\times k^m} \backslash \mathcal{H}$.

(ii) $\mathcal{L}_p$ the set of sub-Nash equilibriums, which are obtained from p-th iteration.

(iii) $L_p \in \mathcal{L}_p$, the strategy profile chosen from $\mathcal{L}_p$ for $(p+1)$-th iteration.

(iv) N_{L_p}: The neighborhood of L_p,

$$N_{L_p} := \{\, L \in \mathcal{L}_{k\times k^{mu}} |\, d(L, L_p) \leq \varepsilon \}.$$

The default ϵ is $\epsilon = 1$.

Algorithm 5.2:

- Step 0. Choose an initial strategy profile L_0, set $\mathcal{H} = \{L_0\}$.
- Step 1. On the neighbor N_{L_0} of L_0 search sub-Nash equilibrium(s), denoted as

$$\mathcal{L}_1 = \{L_1^1, L_1^2, \cdots, L_1^{k_1}\}.$$

  - If $L_0 \in \mathcal{L}_1$, choose L_0 as a local sub-Nash equilibrium (the solution) and stop.
  - Else, if $\mathcal{L}_1 \cap \mathcal{H}^c = \emptyset$, no local sub-Nash equilibrium is found (the algorithm fails) and stop.
  - Else, choose $L_1 \in \mathcal{L}_1 \cap \mathcal{H}^c$, and add L_1 to $\mathcal{H}$.
- Step 2. On the neighbor N_{L_1} of L_1 search sub-Nash equilibrium(s), denoted as

$$\mathcal{L}_2 = \{L_2^1, L_2^2, \cdots, L_2^{k_2}\}.$$

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on December 31,2025 at 08:10:04 UTC from IEEE Xplore. Restrictions app

- If $L_1 \in \mathcal{L}_2$, choose L_1 as a local sub-Nash equilibrium (the solution) and stop.
- If $\mathcal{L}_2 \cap \mathcal{H}^c = \emptyset$, go back to Step 1.
- Else, choose $L_2 \in \mathcal{L}_1 \cap \mathcal{H}^c$, and add L_2 to $\mathcal{H}$.

- $\cdots$
- Step p. On the neighbor $N_{L_{p-1}}$ of L_{p-1} search sub-Nash equilibrium(s), denoted as

$$\mathcal{L}_p = \{L_p^1, L_p^2, \cdots, L_p^{k_p}\}.$$

  - If $L_{p-1} \in \mathcal{L}_p$, choose L_{p-1} as a local sub-Nash equilibrium (the solution) and stop.
  - Else, if $\mathcal{L}_p \cap \mathcal{H}^c = \emptyset$, go back to Step p-1.
  - Else, choose $L_p \in \mathcal{L}_p \cap \mathcal{H}^c$, and add L_p to $\mathcal{H}$.

- $\cdots$

Remark 5.3: 1) Since $|\mathcal{L}_{k\times k^\mu}| = k^{(k^\mu)} < \infty$, the algorithm will terminate somewhere.

2) Only when the chosen optimal strategy profiles $L_0, L_1, \cdots$, form a cycle, the algorithm fails. Otherwise, a local sub-Nash equilibrium can be obtained. Theoretically, we have no reason to claim that the algorithm will never fail. But in numerical computations, we did not have experience of failing.

We give some examples to depict it.

Example 5.4: Consider the infinitely repeated game of prisoners' dilemma, the payoff bi-matrix is

TABLE I
PAYOFF BI-MATRIX

$P_1 \backslash P_2$	1	2
1	3,3	0,5
2	5,0	1,1

Choose $L_0 = \delta_4[1\ 1\ 1\ 3]$, using Algorithm 5.2, we have

$$\begin{array}{ll} L_1 = \delta_4[1\ 4\ 1\ 3], & L_7 = \delta_4[4\ 4\ 4\ 2] \\ L_2 = \delta_4[1\ 3\ 1\ 3], & L_8 = \delta_4[1\ 4\ 4\ 2] \\ L_3 = \delta_4[1\ 3\ 4\ 3], & L_9 = \delta_4[2\ 4\ 4\ 2] \\ L_4 = \delta_4[3\ 3\ 4\ 3], & L_{10} = \delta_4[2\ 4\ 4\ 4] \\ L_5 = \delta_4[3\ 3\ 4\ 2], & L_{11} = \delta_4[1\ 4\ 4\ 4]. \\ L_6 = \delta_4[3\ 4\ 4\ 2], & \end{array}$$

The algorithm terminates at $k = 11$, L_{11} is a locally Nash equilibrium, which is also a 1-memory Nash equilibrium [9].

The algorithm can not always find a Nash equilibrium, for example, let $L_0 = \delta_4[2\ 1\ 2\ 3]$, then we can find a locally Nash equilibrium $L = \delta_4[1\ 4\ 2\ 1]$ which is not Nash equilibrium.

VI. Optimization of Mixed-value Logical Networks

The dynamics of a mixed-value logical control network is expressed as

$$\begin{cases} x_1(t+1) = f_1(x_1, \cdots, x_n, u_1, \cdots, u_m) \\ x_2(t+1) = f_2(x_1, \cdots, x_n, u_1, \cdots, u_m) \\ \cdots \\ x_n(t+1) = f_n(x_1, \cdots, x_n, u_1, \cdots, u_m), \end{cases} \tag{23}$$

where $x_i \in \mathcal{D}_{k_i}$, $i = 1, \cdots, n$; $u_j \in \mathcal{D}_{r_j}$, $j = 1, \cdots, m$; f_i, $i = 1, \cdots, n$ are logical functions. It is briefly denoted as

$$\begin{array}{l} X(t+1) = F(X(t), U(t)), \\ X(t) \in \prod_{i=1}^n \mathcal{D}_{k_i}, \ \ U(t) \in \prod_{j=1}^m \mathcal{D}_{r_j}. \end{array} \tag{24}$$

Let $x(t) := \ltimes_{i=1}^n x_i(t)$, $u(t) := \ltimes_{i=1}^m u_i(t)$, $k := \prod_{i=1}^n k_i$, and $r := \prod_{j=1}^m r_j$. We can equivalently express (23) (equivalently, (24)) in an algebraic form as [5]

$$x(t+1) = Lu(t)x(t), \quad x \in \Delta_k, \ u \in \Delta^r \tag{25}$$

where $L \in \mathcal{L}_{k\times k+r}$ is also called the transition matrix.

Assume the performance criterion is

$$J(u) = \overline{\lim_{T\to\infty}} \frac{1}{T} \sum_{t=1}^{T} c(x(t), u(t)), \tag{26}$$

where c is a function from $\mathcal{D}^{k+r}$ to $\mathbb{R}$. This criterion may be considered as for a particular player in game theory.

Definition 6.1: Consider the mixed-value control network (23). Assume $x_0 = x(0)$ is given. The optimization problem with respect to criterion (26) is finding $\{u^*(t) | t = 0, 1, 2, \cdots\}$, such that

$$J(u^*) \geq J(u), \quad \forall u. \tag{27}$$

We cite some basic results from [18], [24], [23], which are the starting point of the work in this section.

The following result is from [18] and extended to general case by [24], [23].

Theorem 6.2: There exists an optimal trajectory, corresponding to the optimal control, which is periodic on state-control product space.

The following result is from [24].

Theorem 6.3: The optimal control can be expressed as

$$\begin{cases} u_1(t+1) = g_1(x_1(t), \cdots, x_n(t), u_1(t), \cdots, u_m(t)) \\ u_2(t+1) = g_2(x_1(t), \cdots, x_n(t), u_1(t), \cdots, u_m(t)) \\ \vdots \\ u_m(t+1) = g_m(x_1(t), \cdots, x_n(t), u_1(t), \cdots, u_m(t)). \end{cases} \tag{28}$$

Remark 6.4: According to Theorem 6.3, we can combine (28) with (23) to get an overall system. It is easy to see that the overall system (23)-(28) is the same as (4) with $\mu = 1$. But the control (strategy) considered in this section is initial-value-depending.

Recall the system (23). Its algebraic form is expressed as

$$x(t+1) = Lx(t)u(t), \tag{29}$$

where $L \in \mathcal{L}_{k\times kr}$. Similarly, the algebraic form of (28) is denoted as

$$u(t+1) = Gx(t)u(t), \tag{30}$$

where $U \in \mathcal{L}_{r\times kr}$.

Multiplying both sides of (29) and (30) together yields

$$\begin{array}{rcl} x(t+1)u(t+1) & = & Lx(t)u(t)Gx(t)u(t) \\ & = & L(I_{kr} \otimes G)x(t)u(t)x(t)u(t) \\ & = & L(I_{2^{n+m}} \otimes G)\Phi x(t)u(t), \end{array}$$

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on December 31,2025 at 08:10:04 UTC from IEEE Xplore. Restrictions app

where

$$\Phi = \mathrm{diag}\left(\delta_{kr}^1, \delta_{kr}^2, \cdots, \delta_{kr}^{kr}\right).$$

Define $w(t) = x(t)u(t)$, then we have

$$w(t+1) = \Psi(G)w(t), \tag{31}$$

For every G, we can calculate $\Psi(G)$, and then find the cycles of (31) for every initial u_0, thus by comparing the criterions we can find the optimal control. But in general, searching all $G \in \mathcal{L}_{r\times kr}$ to find an optimal solution is unrealistic because of the computation complexity. Note that the method in [24] which search the optimal cycle of logical control network also has huge computation complexity. Again, at each step, we look for only a local optimal solution. That is, look for optimal solution G over a neighborhood

$$N_{G_0} := B_\epsilon(G_0) = \{G \in \mathcal{L}_{r\times rk} |\, d(G, G_0) \le \varepsilon\}.$$

Set the default $\epsilon = 1$.

We propose the following algorithm.

Algorithm 6.5:

- Step 1: Choose a $G_0 \in \mathcal{L}_{2^m\times 2^{m+n}}$.
- Step p: Find the best solutions over $N_{G_{p-1}}$, say,

$$\mathcal{G}_p = \{(G_p^1, u_0^{p,1}), \cdots, (G_p^{k_p}, u_0^{p,k_p})\}.$$

If there exists a $G_p^i = G_{p-1}$, stop. Taking $(G_{p-1}, u_0^{p,i})$ as the local optimal control.

Else, choose $G_p = G_p^1$, continue.

Example 6.6: Consider a Boolean network

$$x(t+1) = Lx(t)u(t),$$

where

$$L = \delta_2[1\ 2\ 2\ 1].$$

Assume

$$c(x(t), u(t)) = u'(t)\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} x(t)$$

and $x_0 = \delta_2^2$, $L = \delta_2[1\ 2\ 2\ 1]$, $x_0 = \delta_2^2$. Choose $G_0 = \delta_2[1\ 2\ 2\ 1]$, using algorithm 6.5 we can get that in step 1

$$\mathcal{G}_1 = \{(\delta_2[1\ 2\ 2\ 2], \delta_2^1), (\delta_2[1\ 2\ 2\ 2], \delta_2^2)\}.$$

We choose $G_1 = \delta_2[1\ 2\ 2\ 2]$, then

$$\begin{aligned}\mathcal{G}_2 = \{&(\delta_2[1\ 2\ 2\ 2], \delta_2^1), (\delta_2[1\ 2\ 2\ 2], \delta_2^2),\\ &(\delta_2[2\ 2\ 2\ 2], \delta_2^1), (\delta_2[2\ 2\ 2\ 2], \delta_2^2)\}.\end{aligned}$$

Thus, $(\delta_2[1\ 2\ 2\ 2], \delta_2^1)$ and $(\delta_2[1\ 2\ 2\ 2], \delta_2^2)$ are local optimal controls. We can check that they are also optimal controls.

VII. Conclusion

In this paper a metric was introduced to the set of strategy profiles of a dynamic game with finite memory. The geometric meaning of the metric was explained. Using it, the concept of local Nash/subNash equilibrium(s) was introduced and the geometric meaning was revealed. When the initial-value-independent strategy profiles for dynamic games were considered, a hill climbing algorithm was introduced to find the local Nash/sub-Nash equilibrium(s). As another application, the optimization of mixed-value logical control systems was investigated, and the algorithm is proposed to get local "best" initial-value-dependent strategy.

References

[1] T. Akutsu, S. Miyano, S. Kuhara, Inferring qualitative relations in genetic networks and metabolic pathways, *Bioinformatics*, Vol. 16, 727-73, 2000.

[2] R. Albert, A-L Barabasi, Dynamics of complex systems: scaling laws or the period of Boolean networks, *Phys. Rev. Lett.*, Vol. 84, 5660-5663, 2000.

[3] D. Cheng, Semi-tensor product of matrices and its applications — A survey, *ICCM 2007*, Vol. 3, 641-668, 2007.

[4] D. Cheng, Input-state approach to Boolean networks, *IEEE Trans. Neural Network*, Vol. 20, No. 3, 512-521, 2009.

[5] D. Cheng, H. Qi, Controllability and observability of Boolean networks, *Automatica*, Vol. 45, No. 7, 1659-1667, 2009.

[6] D. Cheng, Z. Li, H. Qi, Realization of Boolean control networks, *Automatica*, Vol. 46, No. 1, 62-69, 2010.

[7] D. Cheng, H. Qi, A linear representation of dynamics of Boolean networks, *IEEE Trans. Aut. Contr.*, Vol. 55, No. 10, 2010. (to appear).

[8] D. Cheng, H. Qi, Z. Li, J. Liu, Stability and stabilization of Boolean networks, *Int. J. Robust Nonlin. Contr.*, 2010. (to appear)

[9] D. Cheng, Y. Zhao, Z. Li, Sub-Nash solution to infinitely repeated games. (submitted to *Int. J. Game Theory*).

[10] C. Farrow, J. Heidel, H. Maloney, J. Rogers, Scalar equations for synchronous Boolean networks with biological applications, *IEEE Trans. Neural Networks*, Vol. 15, No. 2, 348-354, 2004.

[11] R. Gibbons, *A Primer in Game Theory*, FT Prentice Hall, London, 1992.

[12] S. Huang, I. Ingber, Shape-dependent control of cell growth, differentiation, and apotosis: Switching between attractors in cell regulatory networks, *Exp. Cell Res.*, Vol. 261, 91-103, 2000.

[13] S. Huang, Regulation of cellular states in mammalian cells from a genomewide view, in *Gene Regulation and Metabolism*, J. Collado-Vodes and R. Hofestadt, Eds. Cambridge, MA:MIT Press, 181-220, 2002.

[14] S.A. Kauffman, Metabolic stability and epigenesis in randomly constructed genetic nets, *J. Theoretical Biology*, Vol. 22, 437-467, 1969.

[15] S.A. Kauffman, *The Origins of Order: Self-organization and Selection in Evolution*, Oxford University Press, New York, 1993.

[16] S.A. Kauffman, *At Home in the Universe*, Oxford University Press, New York, 1995.

[17] Z. Li, D. Cheng, Algebraic Approach to Dynamics of Multi-valued Networks, *Int. J. Bifurcat. Chaos*, Vol. 20, No. 3, 561-582, 2010.

[18] Y. Mu, L. Guo, Optimization and identification in a non-equilibrium dynamic game, *Proc. CDC-CCC'09*, 5750-5755, 2009.

[19] M. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, John Wiley & Sons, Inc., New York, 1994.

[20] B. Samuelsson, C. Troein, Superpolynomial growth in the number of attractots in kayffman networks, *Phys. Rev. Lett.*, Vol. 90, 90098701, 2003.

[21] I. Shmulevich, R. Dougherty, S. Kim, W. Zhang, Probabilistic Boolean neworks: A rule-based uncertainty model for gene regulatory networks, *Bioinformatics*, Vol. 2, No. 18, 261-274, 2002.

[22] F. Robert, *Discrete Iterations, A Metric Study*, Translated by J. Rokne, Springer-Verlag, Berlin, 1986.

[23] Y. Zhao, D. Cheng, Optimal control of mix-valued logical control systems, *Proc. CCC'10*, 1618-1623, 2010.

[24] Y. Zhao, D. Cheng, Optimal control of logical control networks, preprint. (submitted for pub)

thorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on December 31,2025 at 08:10:04 UTC from IEEE Xplore. Restrictions app